Caso 3 – Tecnología e Infraestructura de Computo

Gabriel Felipe Mendez Sarmiento – 202512110

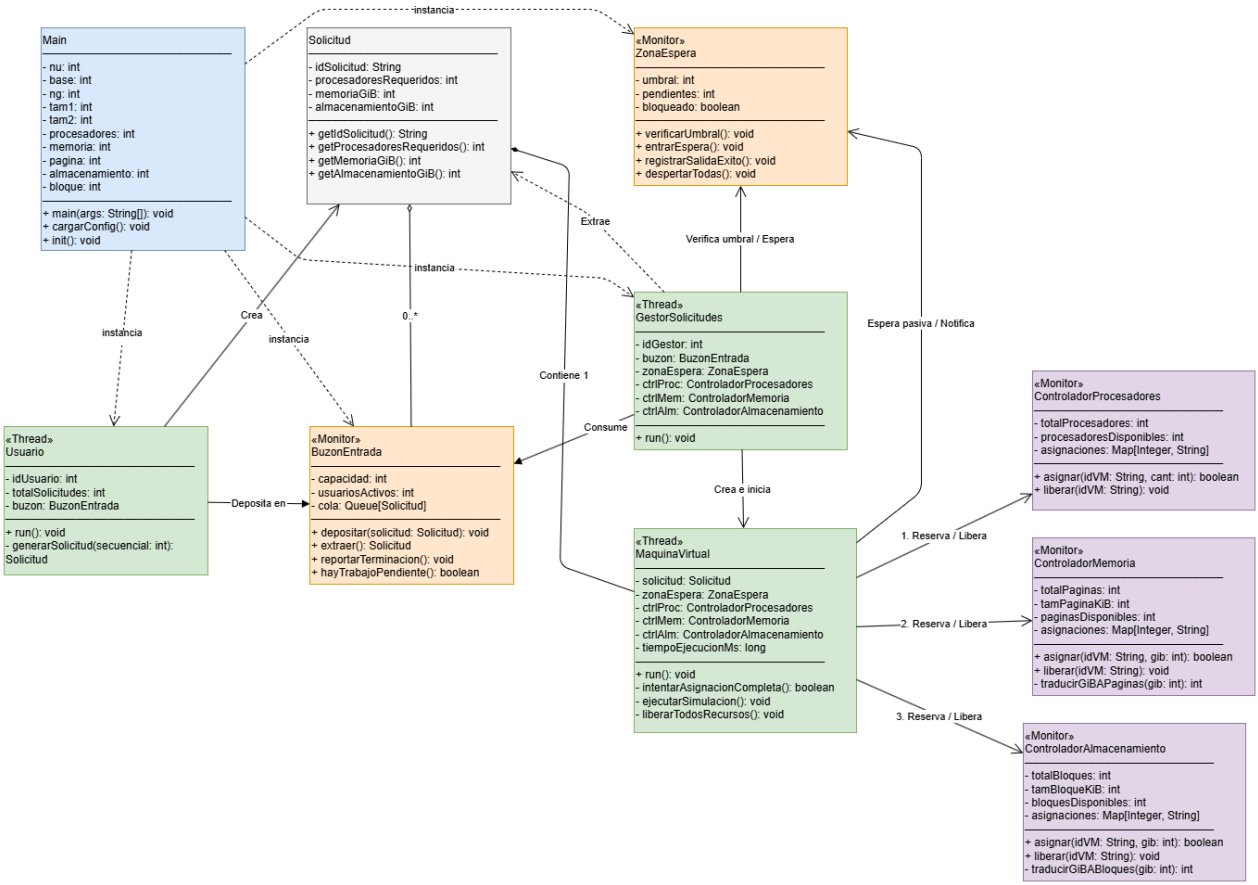
Moisés Ortiz Espinosa – 202510971

Diseño

El sistema sigue un modelo Productor-Consumidor. Los Usuarios (Productores) inyectan solicitudes al Buzon que está limitado. Los Gestores (Consumidores) consumen las solicitudes del Buzon y las despachan para lanzar MVs como hilos independientes.

Cada MV es un proceso con 3 recursos compartidos que adquiere en orden estricto y que poseen Controladores independientes. Si una MV no obtiene sus 3 recursos respectivos, se aborta la MV liberando cualquier otro recurso que se le haya alocado y se suspende en la Zona de Espera. Una MV que termina su ciclo despierta a las de la Zona de Espera para que reintenten obtener sus recursos.

El Gestor supervisa la congestión deteniendo la entrada de solicitudes si se supera el umbral de la Zona de Espera.



Preguntas

1. ¿Qué problema resuelve el umbral, con la espera total del gestor hasta procesar las pendientes?

El umbral ayuda a que las MVs que ya están esperando tengan prioridad. Si el gestor siguiera creando MVs empezarían a competir con las que están en espera y podrían quitarles recursos y las que están dormidas nunca se ejecutarían y buscarían sus recursos. Adicional, la zona de espera comenzaría a crecer sin control, empezarían crearse muchos threads a la vez que no funcionaría en el sistema. Al llegar al umbral, el gestor no admite ninguna solicitud hasta que se procesen todas las pendientes.

2. ¿Por qué es importante que todas las MVs reserven en el mismo orden?

   Por el bloqueo mutuo, si una MV espera un recurso que otra MV ya tiene y viceversa ninguna avanza pues se quedan esperando el recurso de la otra. En cambio, si se mantiene el orden una MV que tiene memoria ya no necesita procesador y con el orden fijo, quien espera un recurso solo espera a quien tiene uno posterior en el orden. Como el que tiene disco no espera a nadie, no se forma una espera circular.

3. ¿Por qué hay que liberar lo ya reservado si no se completa la reserva?

   La MV no quedaría completa en caso de que no se complete la alocación de recursos, entonces no tendría sentido usar una MV incompleta para el usuario pues necesita de todos los recursos para ejecutarse, adicional así no se usan recursos adicionales en MVs sin usar y se liberan para que otras VM puedan reservar el recurso correctamente. Además, liberar todo evita que una MV espere teniendo recursos retenidos.

4. Al liberar recursos, ¿se despierta una MV o todas, y por qué?

   Al liberar recursos se deberían despertar todas con notifyAll, debido a que todas entran en condición de carrera y revisan si les alcanza para el recurso, en caso de que no les alcance se vuelven a dormir y esperan a volver a ser despertadas. Así no hay problemas en que se despierten todas aunque tenga un costo adicional. Despertar solo a una significaría despertar a una MV que podría no caber con lo liberado, en ese caso, se volvería a dormir y las que sí pueden entrar seguirían dormidas con recursos libres. Con los recursos sin reservar para la MV reservada solo se le avisa y vuelve a intentar la reserva desde el inicio, lo que libera una MV que puede alcanzar varias.

Parejas

Pareja	Comparten	Que falla sin sincronizar	Quien espera	Tipo de espera	Quien despierta

Gestor <-> Buzón	Solicitudes, conador de usuarios activos	Dos gestores sacan la misma solictud, o sacan de un buzon vacio, se debe hacer en una sola operacion con synchronized. El gestor puede terminar antes de tiempo si ve un buzon vacio y 0 usuarios activos, dejando una solictud sin procesaar, debe evaluarse al tiempo con synchronized	El gestor si el buzón esta vacío	Semiactiva	Con espera semiactiva , nadie espera al gestor, el mismo vuelve a revisar
Gestor <-> Zona de espera	Numero de MV's en espera	El gestor puede leer un conteo desactualizado y admitir mas solicutdes de las pendientes. Si el gestor revisa los pendientes y se duerme por separado, puede perder un notify() que llega justo en medio. Debe revisar y hacer wait() dentro del mismo synchronized.	MVs dormidas y el gestor cuando el conteo llega al umbral	pasiva	La ultima MV que sale de la zona. A las MVs dormidas las despierta la MV que termina y libera recursos
MV<-> Controladores	Estado de recursos	Sin sincronización ambas MVs pueden verlo libre y		pasiva	La MV que espera la despierta otra MV que termina y libera recursos

		quedarse con él. Con synchronized, una entra primero y la otra lo encuentra ocupado. Además, reservar N recursos debe ser una sola operación atómica: revisar que hay N libres y asignarlos dentro del mismo método synchronized.			
MV<-> Zona de espera	Conteo de MVs en espera	Señal perdida, una MV falla al reservar, liberar y antes de volver a dormirse otra MV libera y avisa. Nadie estaba dormido, el aviso se pierde y la MV queda dormida con recursos ya libress	La MV	pasiva	La MV que termina y libera con notifyAll
Usuario<-> Buzón	la cola de solicitudes	dos usuarios escriben en la misma posición y se pierde una solicitud; o se escribe con el buzón lleno	el usuario, si el buzón está lleno	pasiva	el gestor, cuando saca una solicitud (notify/notifyAll)

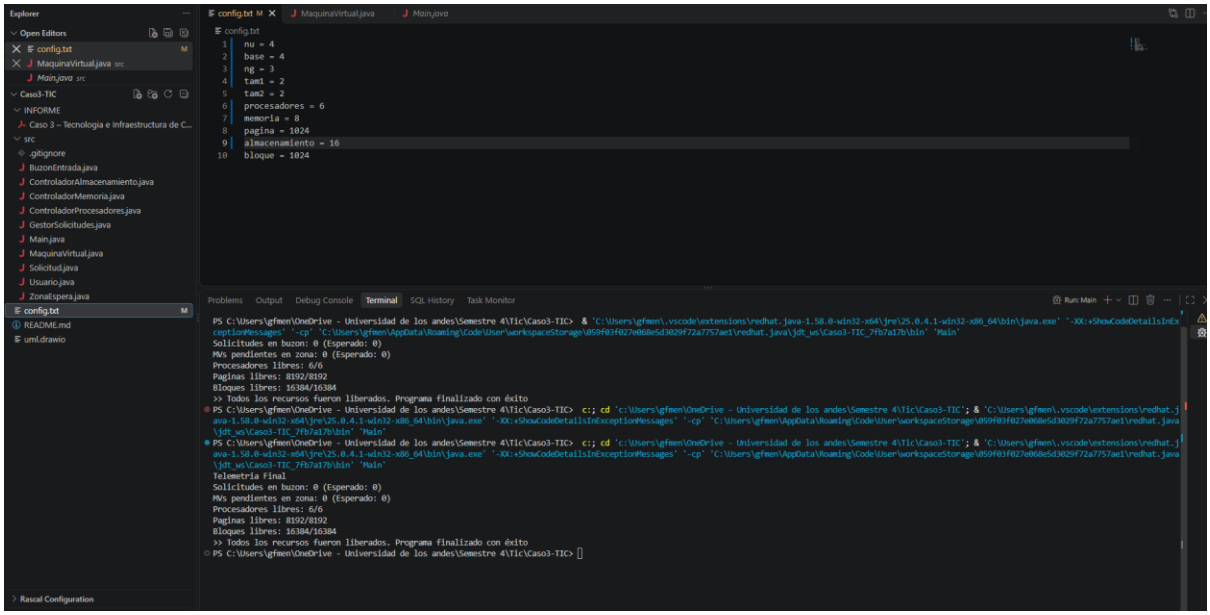
Implementación de la Espera "Total" del Gestor

La Espera para el umbral >= se implementa usando el contador del umbral como condición que otorga el valor a un centinela "bloqueado", tal que si pendientes >= umbral entonces

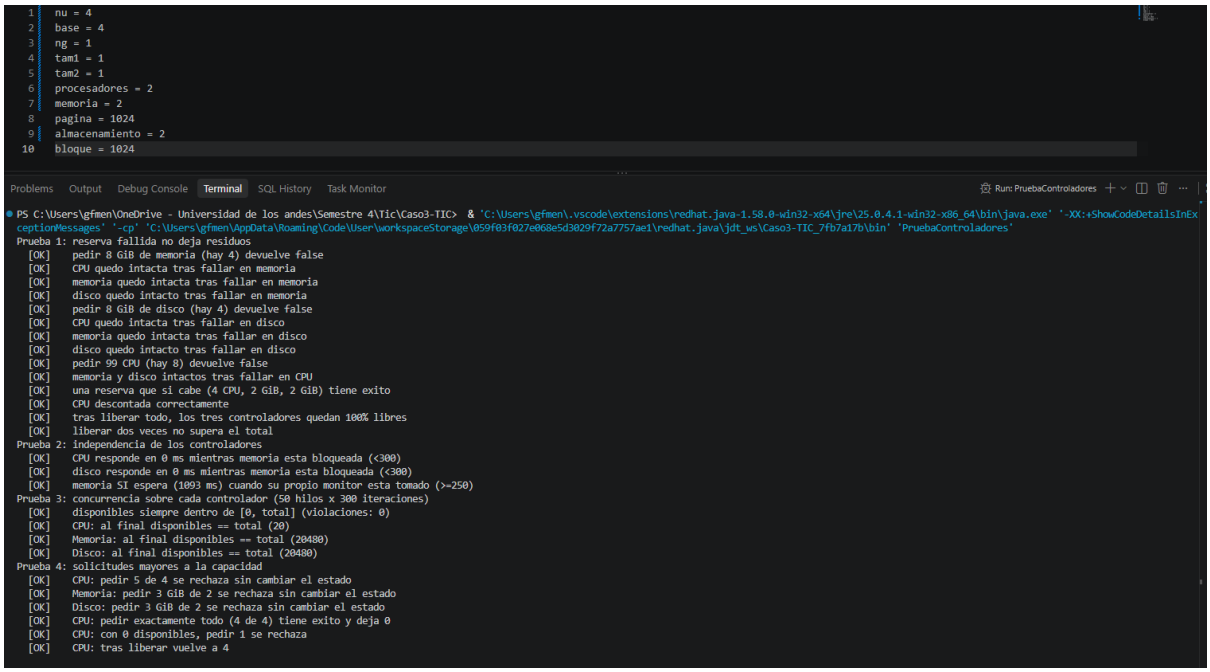
bloqueado. Y el centinela va en un while de un syncronized(ZonaEspera) que suspende el Gestor mientras el centinela este activo. De modo que el Gestor no consuma CPU o ingrese más solicitudes mientras se vuelve a vaciar la zona de Espera.
Así, si una MV termina, entra a syncronized(zonaEspera) y decrementa pendientes. Y que si pendientes vuelve a 0, la última MV actualiza pendientes y ejecuta el notifyAll que permite que el Gestor nuevamente evalué su while(bloqueado), no se cumple y ya puede procesar nuevamente solicitudes del Buzón.

Validaciones

- Prevención de DeadLocks



- Controladores Independientes



- Funcionamiento de Umbral de Congestión

config.txt

```
nu = 4
base = 4
ng = 1
tam1 = 1
tam2 = 1
procesadores = 2
memoria = 2
pagina = 1024
almacenamiento = 2
bloque = 1024
```

Problems Output Debug Console Terminal SQL History Task Monitor

Run: Main

```
PS C:\Users\gfmen\OneDrive - Universidad de los andes\Semestre 4\Tic\Caso3-TIC> c:; cd 'c:\Users\gfmen\OneDrive - Universidad de los andes\Semestre 4\Tic\Caso3-TIC'; & 'C:\Users\gfmen\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\gfmen\AppData\Roaming\Code\User\workspaceStorage\059f03f027e068e5d3029f72a7757ae1\redhat.java\jdt_ws\Caso3-TIC_7fb7a17b\bin' 'Main'
[ESPERA] Thread-18 consigue recursos y sale. pendientes=0
[MV] 4-5 EJECUTA con sus 3 recursos
[UMBRAL] Gestor-1 SE DESBLOQUEA: pendientes=0
[GESTOR] Gestor-1 lanza MV 2-3
[GESTOR] Gestor-1 lanza MV 3-2
[ESPERA] Thread-19 entra a la zona. pendientes=1 (umbral=1)
[UMBRAL] Gestor-1 SE BLOQUEA: pendientes=1 >= umbral=1
[ESPERA] Thread-20 entra a la zona. pendientes=2 (umbral=1)
[ESPERA] Thread-19 consigue recursos y sale. pendientes=1
[MV] 2-3 EJECUTA con sus 3 recursos
[ESPERA] Thread-20 consigue recursos y sale. pendientes=0
[MV] 3-2 EJECUTA con sus 3 recursos
[UMBRAL] Gestor-1 SE DESBLOQUEA: pendientes=0
[GESTOR] Gestor-1 lanza MV 4-6
[GESTOR] Gestor-1 lanza MV 3-3
[GESTOR] Gestor-1 lanza MV 2-4
[ESPERA] Thread-21 entra a la zona. pendientes=1 (umbral=1)
[ESPERA] Thread-22 entra a la zona. pendientes=2 (umbral=1)
[UMBRAL] Gestor-1 SE BLOQUEA: pendientes=2 >= umbral=1
[ESPERA] Thread-23 entra a la zona. pendientes=3 (umbral=1)
[ESPERA] Thread-21 consigue recursos y sale. pendientes=2
[MV] 4-6 EJECUTA con sus 3 recursos
[ESPERA] Thread-22 consigue recursos y sale. pendientes=1
[MV] 3-3 EJECUTA con sus 3 recursos
```